

DOCUMENTATION

RegCheck

Project Report

DATE May 22, 2026

Table of Contents

01 1. Executive summary

02 2. Academic context and deliverables

2.1 Relation to subject tool examples

2.2 Original product brief (README)

03 3. Problem, solution, and use cases

3.1 3.1 The problem

3.2 3.2 The solution

3.3 3.3 Personas and use cases

3.4 3.4 Value delivered

04 4. Functional requirements

4.1 4.1 Minimum MVP (from README)

4.2 4.2 Extended requirements (knowledge base / Submit 1)

05 5. Scope and boundaries

5.1 5.1 In scope

5.2 5.2 Out of scope (MVP)

5.3 5.3 Known technical debt

06 6. System architecture

6.1 6.1 High-level components

6.2 6.2 Request lifecycle (backend)

6.3 6.3 Runtime topologies

6.4 6.4 CORS and browser security

07 7. Technology stack

7.1 7.1 Backend

7.2 7.2 Frontend

7.3 7.3 Infrastructure

7.4 7.4 Design decisions

08 8. Repository structure

8.1 8.1 regcheck (frontend)

8.2 8.2 regcheck-backend (API)

09 9. Data model and persistence

9.1 9.1 Entity-relationship overview

9.2 9.2 Table reference

9.3 9.3 Seeding policy

9.4 9.4 Assessment snapshot semantics

10 10. GDPR catalog and rule engine

10.1 10.1 Seeded rules (10)

10.2 10.2 Seeded checklist items (22)

10.3 10.3 Enrichment layer

10.4 10.4 Recommendation algorithm

10.5 10.5 Company profile schema

11 11. API reference

11.1 11.1 Health

11.2 11.2 Authentication

11.3 11.3 GDPR

11.4 11.4 Assessment summary metrics

12 12. Authentication and authorization

12.1 12.1 Signup flow

12.2 12.2 JWT payload

12.3 12.3 Authorization model

12.4 12.4 Password policy (aligned frontend + backend)

13 13. Frontend application

13.1 13.1 Application states

13.2 13.2 Component catalog

13.3 13.3 Live dashboard pattern

13.4 13.4 Theming

14 14. User journeys

14.1 14.1 First-time user

14.2 14.2 Returning user with history

14.3 14.3 Consultant workflow

15 15. Internationalization

15.1 15.1 Stack

15.2 15.2 Namespaces

15.3 15.3 Locales

15.4 15.4 What is not translated

15.5 15.5 Usage pattern

16 16. Report export

16.1 16.1 Mechanisms

16.2 16.2 Report sections (academic alignment)

16.3 16.3 Data sources for export

17 17. Configuration and environment

17.1 17.1 Frontend (regcheck/.env.example)

17.2 17.2 Backend (regcheck-backend/.env.example)

17.3 17.3 Backend settings (REGCHECK_ prefix)

18 18. Development guide

18.1 18.1 Prerequisites

18.2 18.2 First-time setup (two repos)

18.3 18.3 Daily development loop (recommended)

18.4 18.4 Make targets — regcheck (frontend)

18.5 18.5 Make targets — regcheck-backend

18.6 18.6 Git workflow scripts

19 19. Docker and deployment

19.1 19.1 Railway (current production)

19.2 19.2 Docker Compose — regcheck-backend

19.3 19.3 Docker Compose — regcheck (frontend only)

19.4 19.4 Production frontend image

19.5 19.5 Hetzner deployment (academic target)

20 20. Quality assurance and CI

20.1 20.1 Local gates

20.2 20.2 GitHub Actions (per repository)

20.3 20.3 Testing gap

20.4 20.4 Verification practices (from daily reports)

21 21. Documentation map

22 22. Development chronology

22.1 11-05-2026 — Checklist lifecycle and evidence MVP

22.2 12-05-2026 — Sidebar UX

22.3 13-05-2026 — Authentication and tenancy

22.4 14-05-2026 — Internationalization and layout

22.5 15-05-2026 — Progress metrics

22.6 16-05-2026 — Password policy and ownership

22.7 18-05-2026 — Dashboard and evidence URLs

22.8 19-05-2026 — Assessment update and server export

22.9 Post-19-05-2026 — Split repositories and Railway

23 23. Academic compliance checklist

24 24. Roadmap — Practice 2 and beyond

24.1 24.1 Planned functionalities (minimum 5)

24.2 24.2 Performance and scalability

24.3 24.3 Security improvements for the tool

24.4 24.4 Integrations

24.5 24.5 UX and product debt (near term)

24.6 24.6 Roadmap diagram

25 25. Security, risks, and limitations

25.1 25.1 Security controls implemented

25.2 25.2 Risks

25.3 25.3 Legal disclaimer

26 26. Glossary

27 27. Conclusion

RegCheck — Project Report

Generated: 22-05-2026

Repositories: `regcheck` (frontend), `regcheck-backend` (API) — separate Git repos, developed and deployed independently

URLs: <https://github.com/danielfdez17/regcheck>, <https://github.com/danielfdez17/regcheck-backend>

Deployments (Railway): <https://regcheck-production.up.railway.app/>, <https://regcheck-backend-production.up.railway.app/api/docs>

Version: 0.1.0 (MVP)

Status: Active development — GDPR-focused compliance assistant

Primary audience: Developers, reviewers, academic evaluators

Table of contents

1. Executive summary
2. Academic context and deliverables
3. Problem, solution, and use cases
4. Functional requirements
5. Scope and boundaries
6. System architecture
7. Technology stack
8. Repository structure
9. Data model and persistence
10. GDPR catalog and rule engine
11. API reference
12. Authentication and authorization
13. Frontend application
14. User journeys
15. Internationalization
16. Report export

- [17. Configuration and environment](#)
- [18. Development guide](#)
- [19. Docker and deployment](#)
- [20. Quality assurance and CI](#)
- [21. Documentation map](#)
- [22. Development chronology](#)
- [23. Academic compliance checklist](#)
- [24. Roadmap — Practice 2 and beyond](#)
- [25. Security, risks, and limitations](#)
- [26. Glossary](#)
- [27. Conclusion](#)

1. Executive summary

RegCheck (*Rules to actionable list translator*) is a web application that helps organizations understand which regulatory controls apply to them and what concrete actions they must take to demonstrate compliance. The first implementation targets **GDPR** (General Data Protection Regulation), with a rule catalog, contextual recommendations, prioritized checklists, evidence metadata, and exportable audit reports.

The product sits in the **Regulation and Compliance** track of an AI-assisted cybersecurity development practice. It combines:

- A **Python + FastAPI** REST API with SQL persistence (`regcheck-backend`)
- A **React + TypeScript + Vite** single-page application (`regcheck`)
- **Multi-tenant authentication** (organization + user)
- A **deterministic rule engine** that maps company context to recommended controls
- **Assessment snapshots** that preserve checklist state and evidence over time

The MVP is functional end-to-end: users sign up, configure organizational context, generate and save assessments, track checklist progress, attach evidence references, and export HTML, CSV, and Markdown reports. The codebase is split into two repositories: the **frontend** (`regcheck`) ships as a Vite SPA with optional nginx `/api` reverse proxy for production; the **backend** (`regcheck-backend`) is a FastAPI service with PostgreSQL in Docker or SQLite for local development. Both services are deployed on **Railway**; a Hetzner VPS deployment remains on the academic roadmap.

Strategic positioning: RegCheck does not replace legal advice. It operationalizes regulation into checklists that SMEs can execute, while building an evidence trail suitable for internal audits and academic demonstration.

2. Academic context and deliverables

RegCheck aligns with **Practice 1 — Development of an AI-Powered Cybersecurity Tool** (Regulation and Compliance track). The subject document ([Subject_pr1.EN]) defines mandatory deliverables that this project addresses as follows:

Requirement	How RegCheck addresses it
Web dashboard	Authenticated SPA with assessment dashboard, live metrics, checklist editor, and export
GitHub repository	Public/private repo with structured commits, <code>main</code> / <code>develop</code> workflow, CI on push/PR
Hetzner deployment	Planned for academic submission; Railway hosts production frontend + backend today
PDF documentation report	This Markdown report + HTML export + Obsidian knowledge base feed the PDF deliverable
Practice 2 roadmap	Section 24

Relation to subject tool examples

Among the 15 suggested compliance tools, RegCheck is closest to:

- **#02 GDPR/LOPD gap analyzer** — identifies gaps and produces an adaptation plan via checklist
- **#11 Cybersecurity audit simulator** — questionnaire-driven maturity output
- **#05 AI security policy generator** — concrete actions per control (without generative AI in MVP)

The differentiator for RegCheck MVP is **structured rule-to-checklist mapping** with **evidence metadata** and **persisted assessment history**, not a conversational LLM interface (planned for Practice 2).

Original product brief (README)

From the repository README, the one-month planning phases were:

Week	Goal
1	GDPR scope, control structure, basic backend
2	Frontend + functional checklist, working MVP
3	Self-evaluation and scoring logic
4	UX polish, export, final demo

As of 22-05-2026, weeks 1–2 objectives are met; week 3 scoring is partial (done/total metrics exist; formal scoring model pending); week 4 export is largely done (HTML client-side, CSV/Markdown server-side, assessment PATCH for edits); UX polish and Hetzner migration remain in progress.

3. Problem, solution, and use cases

3.1 The problem

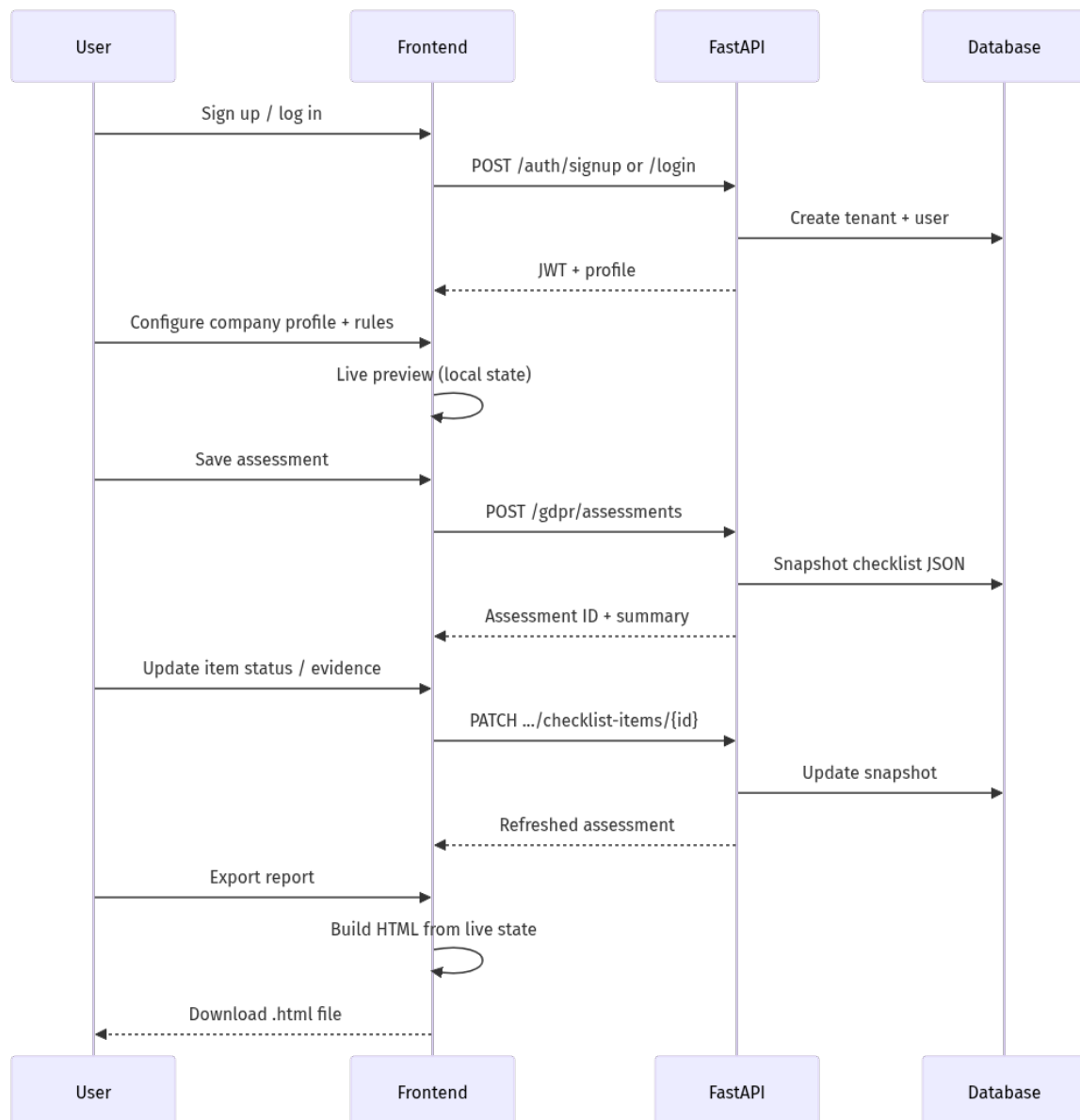
Organizations face three recurring pain points when approaching GDPR:

1. **Interpretation gap** — Legal text does not translate directly into engineering or HR tasks.
2. **Resource gap** — SMEs lack dedicated Data Protection Officers or compliance teams.
3. **Evidence gap** — Auditors require proof of implementation, not only policy documents.

Traditional approaches (spreadsheets, generic templates, expensive consultants) are either too abstract or too costly for small teams.

3.2 The solution

RegCheck implements a **guided intake** → **rule recommendation** → **checklist generation** → **lifecycle tracking** → **evidence linking** → **export** pipeline:



3.3 Personas and use cases

Persona	Goal	RegCheck flow
Startup founder	Know minimum GDPR steps	Select SME profile → auto-recommended rules → high-priority checklist
HR lead	Employee data governance	Select HR department → employee data rules → evidence for access policies
DevSecOps engineer	Secure SDLC evidence	Enable software development → DevSecOps controls → CI gate evidence URLs

Persona	Goal	RegCheck flow
Cyber/SOC analyst	Monitoring and IR	Service description mentions SOC → incident response checklist
Consultant	Repeatable client intake	New tenant per client → multiple saved assessments → HTML export per engagement

3.4 Value delivered

Output	Business value
Prioritized checklist	Focus effort on high-impact controls first
Concrete actions	Each item includes <i>what to implement</i> , not only regulatory jargon
Evidence prompts	Each item states <i>what proof</i> auditors typically expect
Status tracking	Visual progress (done / total) for management reporting
Export	Single artifact for stakeholders, instructors, or auditors

4. Functional requirements

4.1 Minimum MVP (from README)

#	Requirement	Status	Implementation notes
1	Rule selector (GDPR)	Done	<code>GET /gdpr/rule-selector</code> ; 10 rule options in seed
2	Basic rule engine	Done	Keyword, department, framework, and boolean toggles in <code>catalog.py</code>
3	Checklist generation	Done	<code>POST /gdpr/checklists</code> preview; <code>POST /gdpr/assessments</code> persist
4	Status lifecycle	Done	<code>pending</code> , <code>in_progress</code> , <code>done</code> via PATCH

#	Requirement	Status	Implementation notes
5	Export (PDF/CSV/Markdown)	Partial	HTML export (print-to-PDF); CSV/Markdown via <code>GET /gdpr/assessments/{id}/export?format=</code> ; native PDF not implemented

4.2 Extended requirements (knowledge base / Submit 1)

From `[[00-index]]`, the application must collect:

Input	Purpose
Company type	Context for recommendations (startup, SME, enterprise, public sector)
Department types	Maps to rule hints (HR, development, cyber, satellites, operations)
Service description	Free text; keyword matching (cyber, cloud, web, satellite, etc.)
Framework selection	GDPR primary; ISO 27001 optional in catalog
Operational toggles	Cloud, physical buildings, remote VPN

And return:

Output	Status
Compliance checklist	Done
Priority classification	Done (high / medium / low per item)
Concrete actions	Done (<code>concrete_action</code> enrichment)
Evidence repository	Done (metadata: label, URL, notes)
Dynamic follow-up questions	Done (sidebar toggles drive recommendations)

5. Scope and boundaries

5.1 In scope

- GDPR domain mode with seeded catalog (rules + checklist templates)

- Authenticated multi-tenant usage
- Per-user assessment ownership and history (up to 50 items per list request)
- Live UI preview without mandatory API round-trips for checkbox changes
- Bilingual UI shell (English + Spanish)
- Dockerized frontend and backend (separate Compose files per repo)
- Local developer workflow: frontend on :3001, backend on :8000 with SQLite
- Railway production deployment for both services

5.2 Out of scope (MVP)

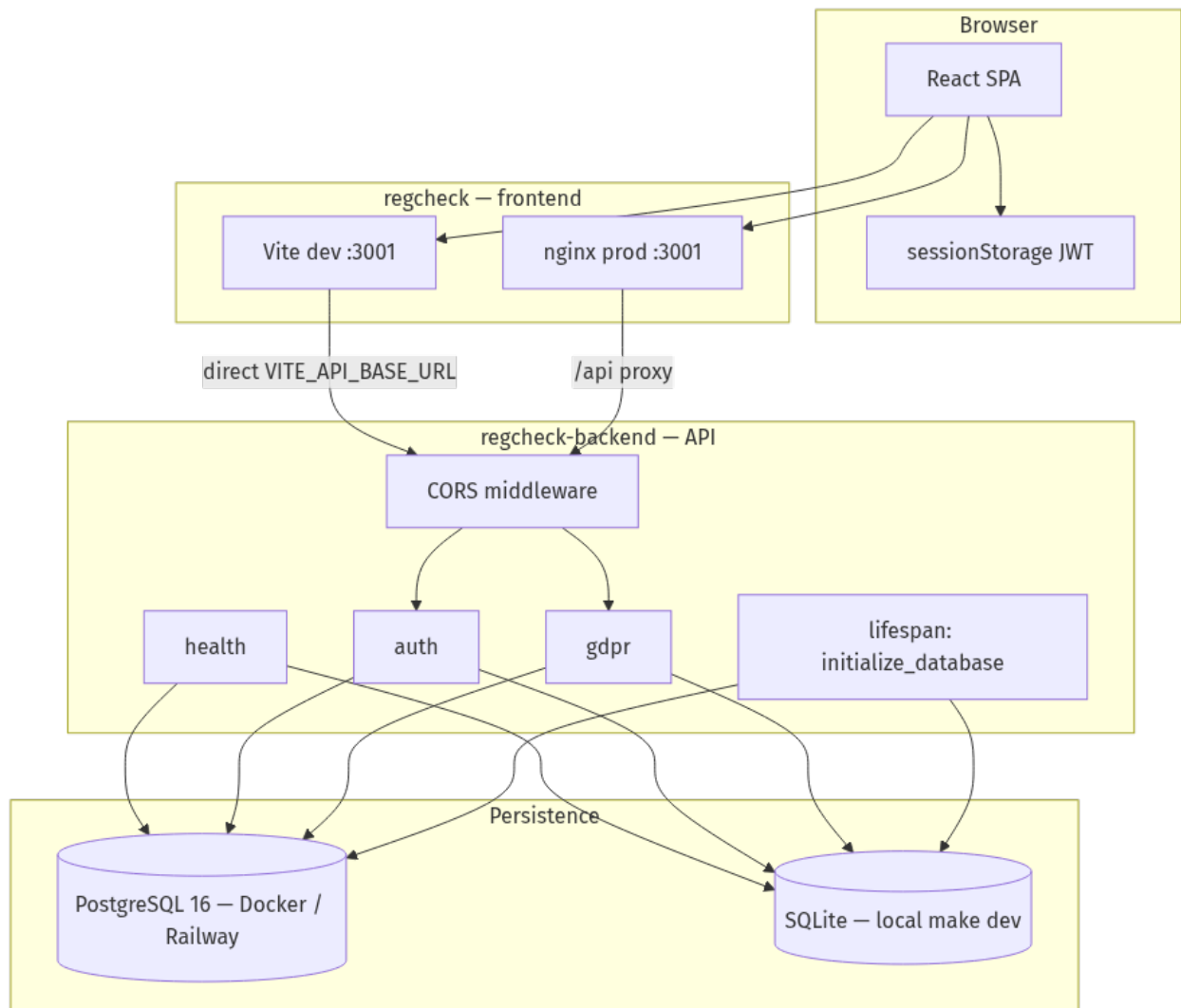
Item	Rationale
File upload for evidence	Metadata/URLs only; reduces storage and malware risk
LLM chat / natural language policy analysis	Practice 2 enhancement
ISO 27001 full support	Present in seed but marked for removal/hiding in UI
Automated third-party integrations	Explicit non-goal per README
Role-based access within tenant	Single user model per signup; no admin roles yet
Email verification / password reset	Not implemented

5.3 Known technical debt

- **Split repositories:** Frontend and backend must be started from two repos; no single-root Compose for the full stack
- **Dual database story:** Backend Docker uses PostgreSQL; `make dev` in `regcheck-backend` uses SQLite by default
- **Schema migrations:** Alembic listed in requirements; startup uses `create_all` + manual SQLite `ALTER` for `created_by_user_id`
- **API content language:** Checklist titles/descriptions are English from seed; UI is localized separately
- **No automated tests:** CI runs typecheck/lint/build (frontend) or compileall/pylint (backend) only
- **Pre-push hook:** `hooks/pre-push` calls `make audit`, which is not defined in either Makefile

6. System architecture

6.1 High-level components



6.2 Request lifecycle (backend)

- 1. Startup** — `lifespan` calls `initialize_database()` : - `SQLModel.metadata.create_all` - SQLite migration guard for `created_by_user_id` - Idempotent seed of domain mode, rules, checklist items
- 2. Request** — FastAPI route → `Depends(get_session)` → business logic → commit
- 3. Auth** — `Depends(get_current_user)` decodes JWT, loads user, returns `CurrentUser`
- 4. GDPR** — `AssessmentOwner(tenant_id, user_id)` scopes all reads/writes

6.3 Runtime topologies

Mode	Commands	Frontend	Backend	Database	Hot reload
Local split (recommended)	<code>make dev</code> in <code>regcheck</code> + <code>make dev</code> in <code>regcheck-backend</code>	:3001 Vite	:8000 Uvicorn -- <code>reload</code>	SQLite <code>regcheck.db</code>	Yes
Frontend Docker	<code>make up</code> in <code>regcheck</code> (backend on host : 8000)	:3001 nginx → host	:8000 on host	Backend DB unchanged	No
Backend Docker	<code>make up</code> in <code>regcheck-backend</code>	—	:8000 Uvicorn	PostgreSQL volume	No
Backend Docker dev	<code>make dev-docker</code> in <code>regcheck-backend</code>	—	:8000 -- <code>reload</code>	PostgreSQL volume	Yes
Railway production	Deploy each repo	SPA + nginx /api proxy	FastAPI + managed Postgres	PostgreSQL	No

Health endpoint: `GET http://localhost:8000/api/v1/health` (or production backend URL)

6.4 CORS and browser security

- **Local dev:** Browser calls backend directly at `VITE_API_BASE_URL` ; CORS allowlist from `REGCHECK_CORS_ORIGINS` and `REGCHECK_FRONTEND_ORIGIN` (default `http://localhost:3001` , `http://127.0.0.1:3001`)
- **Production (Railway / Docker nginx):** Browser uses same-origin `/api/...` ; nginx proxies to the backend — no browser CORS for API calls
- **Railway regex:** `REGCHECK_CORS_ORIGIN_REGEX` defaults to `^https://[a-zA-Z0-9.-]+\..up\.railway\.app$` for cross-origin SPA → API when not proxied
- **Methods:** `GET` , `POST` , `PUT` , `PATCH` , `DELETE` , `OPTIONS` ; headers: `*`
- **Credentials:** `allow_credentials=False` (JWT in `Authorization` header, not cookies)

7. Technology stack

7.1 Backend

Component	Package / tool	Version	Role
Runtime	Python	3.12 (CI)	Language
Framework	FastAPI	0.115.12	HTTP API, OpenAPI, validation
Server	Uvicorn	0.34.0	ASGI server
Settings	pydantic-settings	2.8.1	<code>REGCHECK_*</code> env config
ORM	SQLModel	0.0.24	Models + sessions
Migrations	Alembic	1.15.1	Listed; minimal use in MVP
PostgreSQL driver	psycopg	3.2.10	Docker production DB
Password hashing	bcrypt	4.0.1	Signup/login
JWT	python-jose	3.4.0	Bearer tokens
Linting	Pylint	3.3.5	Backend static analysis

7.2 Frontend

Component	Package	Version	Role
UI library	React	18.3.1	Components
Language	TypeScript	5.6.3	Type safety
Bundler	Vite	6.0.11	Dev server + production build
i18n	i18next, react-i18next	26.x / 17.x	Translations
Icons	react-icons	5.6.0	UI icons
Lint	ESLint + typescript-eslint	8.x	Zero-warning policy

7.3 Infrastructure

Component	Details
regcheck containers	Docker Compose: <code>frontend</code> only (nginx :3001, proxies <code>/api</code> to <code>REGCHECK_API_BASE_URL</code>)
regcheck-backend containers	Docker Compose: <code>postgres</code> + <code>backend</code> (:8000)
Frontend prod image	Multi-stage: Node build → nginx <code>nginx.conf.template</code> + <code>docker-entrypoint.sh</code>
Backend image	Python 3.12-slim, multi-stage venv (<code>backend/Dockerfile</code>)
CI	Separate GitHub Actions per repo (frontend: typecheck/lint/build; backend: compileall/pylint)
Deploy	Railway (<code>railway.json</code> in each repo)
Package manager	pnpm 11.1.1 (frontend)

7.4 Design decisions

Decision	Choice	Why
Monorepo vs split repos	Two Git repos	Frontend and backend deploy and CI independently; docs live in <code>regcheck</code>
SPA vs Next.js	Vite + React	Simpler MVP; README originally mentioned Next.js but tree uses Vite
Production API routing	nginx <code>/api</code> proxy	Same-origin calls on Railway; avoids CORS and exposed backend URL in browser
SQLModel vs raw SQL	SQLModel	FastAPI ecosystem fit, Pydantic integration
JSON snapshots	<code>compliance_assessments.checklist_items</code>	Preserve historical state when catalog templates change
JWT in sessionStorage	Not httpOnly cookies	Simpler SPA auth; XSS is the main threat surface

Decision	Choice	Why
Live preview in frontend	Duplicated checklist logic locally	Instant UX; persistence remains explicit

8. Repository structure

The project uses **two repositories**. Paths below are relative to each repo root.

8.1 regcheck (frontend)

```

regcheck/
├─ frontend/
│  └─ src/
│     │ └─ App.tsx           # auth gate, dashboard vs editor routing
│     │ └─ main.tsx         # React root, providers
│     │ └─ gdpr-playground.tsx # assessment editor, live state, export
│     │ └─ components/      # auth-page, app-shell, home-content, checklist-
item, ...
│     │ └─ auth/            # context, provider, password policy
│     │ └─ hooks/use-home-page-data.ts
│     │ └─ lib/
│     │ │ └─ regcheck-api.ts # typed HTTP client
│     │ │ └─ api-base-url.ts # dev direct / prod same-origin /api
│     │ │ └─ assessment-metrics.ts
│     │ │ └─ report-export.ts # client-side HTML report builder
│     │ └─ evidence-url.ts
│     │ └─ i18n/            # config, locales en/es
│     └─ public/runtime-config.js # production API base (Docker entrypoint)
├─ package.json
├─ vite.config.ts
├─ nginx.conf.template      # /api reverse proxy template
├─ docker-entrypoint.sh
├─ Dockerfile
├─ docs/obsidian/           # Knowledge base + PROJECT-REPORT.md
├─ vendor/scripts/          # Git submodule (git workflow helpers)
├─ .github/workflows/ci.yml # typecheck, lint, build
├─ docker-compose.yml       # frontend service only
├─ docker-compose.dev.yml
├─ railway.json
├─ Makefile
├─ .env.example
└─ README.md

```

8.2 regcheck-backend (API)

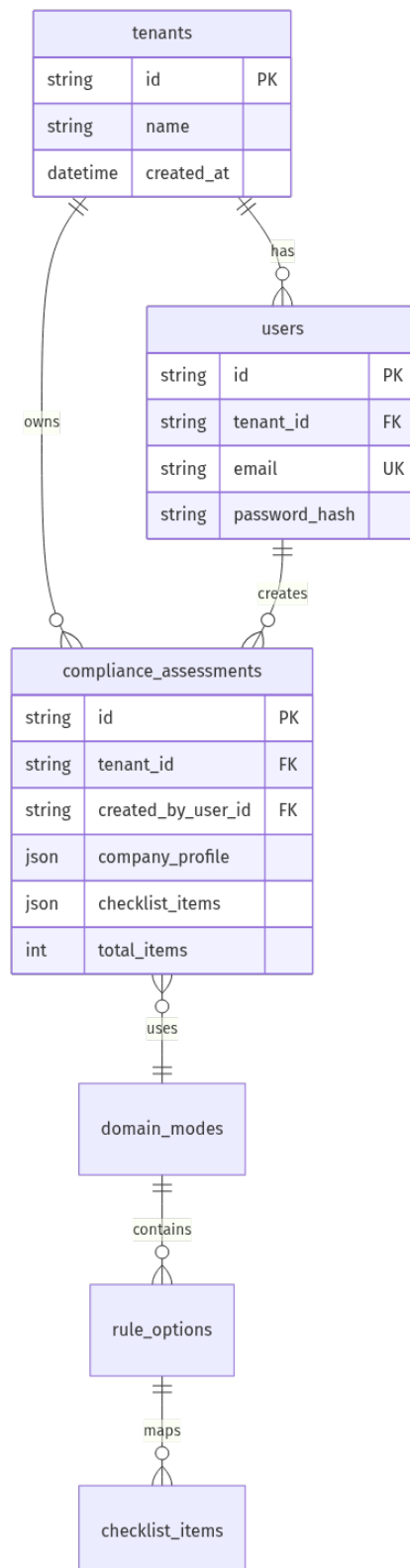
```

regcheck-backend/
├── backend/
│   ├── app/
│   │   ├── main.py                # FastAPI app, CORS, lifespan, router mount
│   │   ├── api/
│   │   │   ├── router.py
│   │   │   └── v1/
│   │   │       ├── auth.py
│   │   │       ├── gdpr.py        # assessments, export, PATCH items
│   │   │       └── health.py
│   │   ├── auth/                  # service, security, dependencies, password_policy
│   │   ├── core/config.py         # Settings (REGCHECK_* + Railway DATABASE_URL)
│   │   ├── db/                   # models.py, session.py (seed, SQLite ALTER guard)
│   │   └── gdpr/
│   │       ├── catalog.py
│   │       ├── assessments.py
│   │       ├── export.py          # CSV/Markdown builders
│   │       ├── seed_data.py
│   │       └── schemas.py
│   ├── requirements.txt
│   └── Dockerfile
├── .github/workflows/ci.yml        # compileall, pylint
├── docker-compose.yml              # postgres + backend
├── docker-compose.dev.yml
├── railway.json
├── Makefile
├── .env.example
└── README.md

```

9. Data model and persistence

9.1 Entity-relationship overview



9.2 Table reference

TENANTS

Column	Type	Description
<code>id</code>	string (PK)	UUID hex
<code>name</code>	string	Enterprise name from signup
<code>created_at</code>	datetime	UTC timestamp

USERS

Column	Type	Description
<code>id</code>	string (PK)	UUID hex
<code>tenant_id</code>	string (FK)	Owning organization
<code>first_name</code> , <code>last_name</code>	string	Display name
<code>email</code>	string (unique)	Normalized to lowercase on signup/login
<code>password_hash</code>	string	bcrypt hash
<code>created_at</code>	datetime	UTC timestamp

DOMAIN_MODES

Column	Type	Description
<code>id</code>	string (PK)	e.g. <code>gdpr</code>
<code>label</code> , <code>description</code>	string	UI/API labels
<code>is_default</code>	bool	Default selector mode

RULE_OPTIONS

Column	Type	Description
<code>id</code>	string (PK)	Stable rule identifier

Column	Type	Description
<code>domain_mode_id</code>	string (FK)	Parent domain
<code>label</code> , <code>description</code>	string	Human-readable rule

CHECKLIST_ITEMS (CATALOG TEMPLATES)

Column	Type	Description
<code>id</code>	string (PK)	e.g. <code>document-processing-activities</code>
<code>rule_id</code>	string (FK)	Parent rule
<code>title</code> , <code>description</code>	string	Control text
<code>priority</code>	string	<code>high</code> , <code>medium</code> , <code>low</code>
<code>status</code>	string	Default <code>pending</code> in catalog

COMPLIANCE_ASSESSMENTS (SNAPSHOTS)

Column	Type	Description
<code>id</code>	string (PK)	Assessment UUID
<code>created_at</code>	datetime	Snapshot time
<code>tenant_id</code>	string (FK)	Organization
<code>created_by_user_id</code>	string (FK, nullable)	Creator scope for list/get/PATCH
<code>domain_mode_id</code>	string (FK)	Usually <code>gdpr</code>
<code>company_profile</code>	JSON	Intake payload at save time
<code>selected_rule_ids</code>	JSON	User-selected rules
<code>recommended_rule_ids</code>	JSON	Engine recommendations
<code>checklist_items</code>	JSON	Full items with status + evidence
<code>total_items</code> , <code>high_priority_items</code> , ...	int	Denormalized metrics (recomputed on read where needed)

9.3 Seeding policy

From [[02-sqlite-schema-and-seeding]]:

- Runs on every application startup via `initialize_database()`
- **Idempotent**: existing IDs are skipped; only missing rows inserted
- Seed source: `backend/app/gdpr/seed_data.py`
- Does not overwrite user assessments

9.4 Assessment snapshot semantics

When a user saves an assessment, the API:

1. Calls `build_gdpr_checklist()` for the current request
2. Computes `AssessmentSummary` (counts, done items, priority breakdown)
3. Persists a **frozen copy** of checklist items as JSON

Subsequent catalog changes do not retroactively alter old assessments. PATCH updates mutate only the snapshot JSON for that assessment ID.

10. GDPR catalog and rule engine

10.1 Seeded rules (10)

Rule ID	Label	Trigger context
<code>personal_data_processing</code>	Personal data processing	Always recommended baseline
<code>devsecops_secure_sdgc</code>	DevSecOps secure SDLC	Development dept; “software” in service text
<code>cloud_security_controls</code>	Cloud security controls	<code>uses_cloud</code> toggle; “cloud” in service text
<code>physical_access_and_video_control</code>	Physical access and cameras	<code>has_physical_buildings</code> toggle

Rule ID	Label	Trigger context
<code>remote_work_vpn_and_password_policy</code>	Remote VPN and password policy	<code>supports_remote_work_vpn</code> toggle
<code>security_monitoring_incident_response</code>	SOC / monitoring / IR	Cyber dept; “cyber”, “soc” in service text
<code>satellite_systems_and_telemetry_security</code>	Satellite and telemetry security	Satellites dept; “satellite”, “photovoltaic”
<code>web_platform_security_and_privacy</code>	Website security and privacy	“web”, “website”, “audit” in service text
<code>employee_data_and_access_governance</code>	Employee data governance	HR department
<code>iso_27001_control_baseline</code>	ISO/IEC 27001 baseline	<code>iso_27001</code> in requested frameworks

10.2 Seeded checklist items (22)

Item ID	Rule	Priority
<code>document-processing-activities</code>	<code>personal_data_processing</code>	high
<code>publish-privacy-policy</code>	<code>personal_data_processing</code>	high
<code>track-retention-periods</code>	<code>personal_data_processing</code>	medium
<code>define-dsr-process</code>	<code>personal_data_processing</code>	high
<code>assign-privacy-owner</code>	<code>personal_data_processing</code>	medium
<code>devsecops-threat-modeling</code>	<code>devsecops_secure_sdgc</code>	high
<code>devsecops-pipeline-security</code>	<code>devsecops_secure_sdgc</code>	high
<code>cloud-iam-hardening</code>	<code>cloud_security_controls</code>	high
<code>cloud-shared-responsibility</code>	<code>cloud_security_controls</code>	medium
<code>physical-badge-governance</code>	<code>physical_access_and_video_control</code>	high
<code>camera-retention-policy</code>	<code>physical_access_and_video_control</code>	medium
<code>vpn-hardening</code>	<code>remote_work_vpn_and_password_policy</code>	high

Item ID	Rule	Priority
<code>password-policy-enforcement</code>	<code>remote_work_vpn_and_password_policy</code>	medium
<code>soc-log-coverage</code>	<code>security_monitoring_incident_response</code>	high
<code>incident-response-tests</code>	<code>security_monitoring_incident_response</code>	high
<code>satellite-telemetry-encryption</code>	<code>satellite_systems_and_telemetry_security</code>	high
<code>satellite-supplier-assurance</code>	<code>satellite_systems_and_telemetry_security</code>	medium
<code>web-consent-management</code>	<code>web_platform_security_and_privacy</code>	high
<code>web-tls-and-security-headers</code>	<code>web_platform_security_and_privacy</code>	medium
<code>hr-access-minimization</code>	<code>employee_data_and_access_governance</code>	high
<code>hr-legal-basis-catalog</code>	<code>employee_data_and_access_governance</code>	medium
<code>iso-risk-register</code>	<code>iso_27001_control_baseline</code>	high
<code>iso-statement-of-applicability</code>	<code>iso_27001_control_baseline</code>	medium

10.3 Enrichment layer

`CHECKLIST_ENRICHMENTS` in `catalog.py` adds human-readable fields for selected high-value controls:

- `concrete_action` — implementation guidance
- `evidence_request` — what to attach for audit

Other items receive generic fallback strings derived from the item title.

10.4 Recommendation algorithm

`recommend_rule_ids(company_profile)` executes in order:

1. Start with `personal_data_processing` (baseline GDPR)
2. Scan `service_description` for `SERVICE_RULE_HINTS` keywords
3. Map each `department_types` entry via `DEPARTMENT_RULE_HINTS`
4. Map each `requested_frameworks` entry via `FRAMEWORK_RULE_HINTS`
5. Append rules for boolean flags: cloud, physical buildings, remote VPN
6. `deduplicate()` preserving order

`merge_rule_ids(selected, recommended, available) :`

- If user selected rules → union with recommendations
- Else if recommendations exist → use recommendations
- Else → all available rules (broad default checklist)

10.5 Company profile schema

```
{
  "company_type": "sme",
  "department_types": ["development", "cyber"],
  "service_description": "SOC monitoring for enterprise clients",
  "requested_frameworks": ["gdpr"],
  "uses_cloud": true,
  "has_physical_buildings": false,
  "supports_remote_work_vpn": true
}
```

Profile options exposed to the UI:

- **Company types:** startup, sme, enterprise, public_sector, other
- **Departments:** hhrr, development, satellites, cyber, operations
- **Frameworks:** gdpr, iso_27001

11. API reference

Base URL: `http://localhost:8000/api/v1`

Authentication: `Authorization: Bearer <jwt>` (required for all routes except health and auth signup/login)

Interactive documentation (Swagger UI): `http://localhost:8000/api/docs`

ReDoc: `http://localhost:8000/api/redoc`

OpenAPI schema: `http://localhost:8000/api/openapi.json`

With the frontend Docker image (`make up` in **regcheck**), nginx proxies `/api/` to the backend configured in `REGCHECK_API_BASE_URL`, so Swagger is available at `http://localhost:3001/api/docs` when the backend is reachable. In local Vite dev, open `http://localhost:8000/api/docs` directly. Use the **Authorize** button with `Bearer <access_token>` after signup or login.

11.1 Health

GET /HEALTH

Response 200:

```
{ "status": "ok" }
```

11.2 Authentication

POST /AUTH/SIGNUP

Body:

```
{
  "first_name": "Ada",
  "last_name": "Lovelace",
  "enterprise": "Analytical Engines Ltd",
  "email": "ada@example.com",
  "password": "SecureP@ssw0rd1"
}
```

Password policy: 12–128 characters; uppercase, lowercase, digit, symbol; no spaces.

Response 201:

```
{
  "access_token": "<jwt>",
  "token_type": "bearer",
  "user": {
    "id": "...",
    "tenant_id": "...",
    "first_name": "Ada",
    "last_name": "Lovelace",
    "email": "ada@example.com",
    "enterprise": "Analytical Engines Ltd"
  }
}
```

Errors: 409 if email exists; 422 if password policy fails.

POST /AUTH/LOGIN

Body: { "email", "password" }

Response 200: Same shape as signup.

Errors: 401 invalid credentials.

GET /AUTH/ME

Response 200: AuthUser object.

POST /AUTH/LOGOUT

Response 204 — client clears token from sessionStorage.

11.3 GDPR

GET /GDPR/RULE-SELECTOR

Returns domain mode, all rules (with checklist_item_ids), and profile_options .

POST /GDPR/CHECKLISTS

Body:

```
{
  "selected_rule_ids": ["personal_data_processing", "cloud_security_controls"],
  "company_profile": { "...": "..." }
}
```

Response: GDPRChecklistResponse with merged rules, checklist items, recommended_rule_ids .

Errors: 400 with unknown_rule_ids if invalid rule IDs.

POST /GDPR/ASSESSMENTS

Same body as checklists. Persists snapshot and returns GDPRAssessmentResponse including assessment_id , created_at , summary .

GET /GDPR/ASSESSMENTS/LATEST

Returns latest assessment for current user or null .

GET /GDPR/ASSESSMENTS?LIMIT=50

Returns AssessmentHistoryResponse with summary cards per assessment.

GET /GDPR/ASSESSMENTS/{ASSESSMENT_ID}

Errors: 404 if not found or not owned by user.

PATCH /GDPR/ASSESSMENTS/{ASSESSMENT_ID}

Same body as `POST /gdpr/assessments`. Regenerates the checklist for an existing snapshot and preserves status/evidence for unchanged item IDs.

Errors: 404 if not found or not owned by user.

GET /GDPR/ASSESSMENTS/{ASSESSMENT_ID}/EXPORT?FORMAT=CSV|MARKDOWN

Returns a downloadable checklist export for the persisted assessment.

Response: `text/csv` or `text/markdown` with `Content-Disposition: attachment`.

Errors: 404 if not found or not owned by user; 422 if `format` is missing or invalid.

PATCH /GDPR/ASSESSMENTS/{ASSESSMENT_ID}/CHECKLIST-ITEMS/{CHECKLIST_ITEM_ID}

Body (partial):

```
{
  "status": "in_progress",
  "evidence_entries": [
    {
      "id": "ev-1",
      "label": "RoPA document",
      "reference_url": "https://intranet.example.com/ropa.pdf",
      "notes": "Approved Q1 2026",
      "created_at": "2026-05-18T10:00:00Z"
    }
  ]
}
```

Response: Full updated `GDPRAssessmentResponse`.

11.4 Assessment summary metrics

`AssessmentSummary` fields:

Field	Meaning
<code>selected_rule_count</code>	Number of rules in assessment

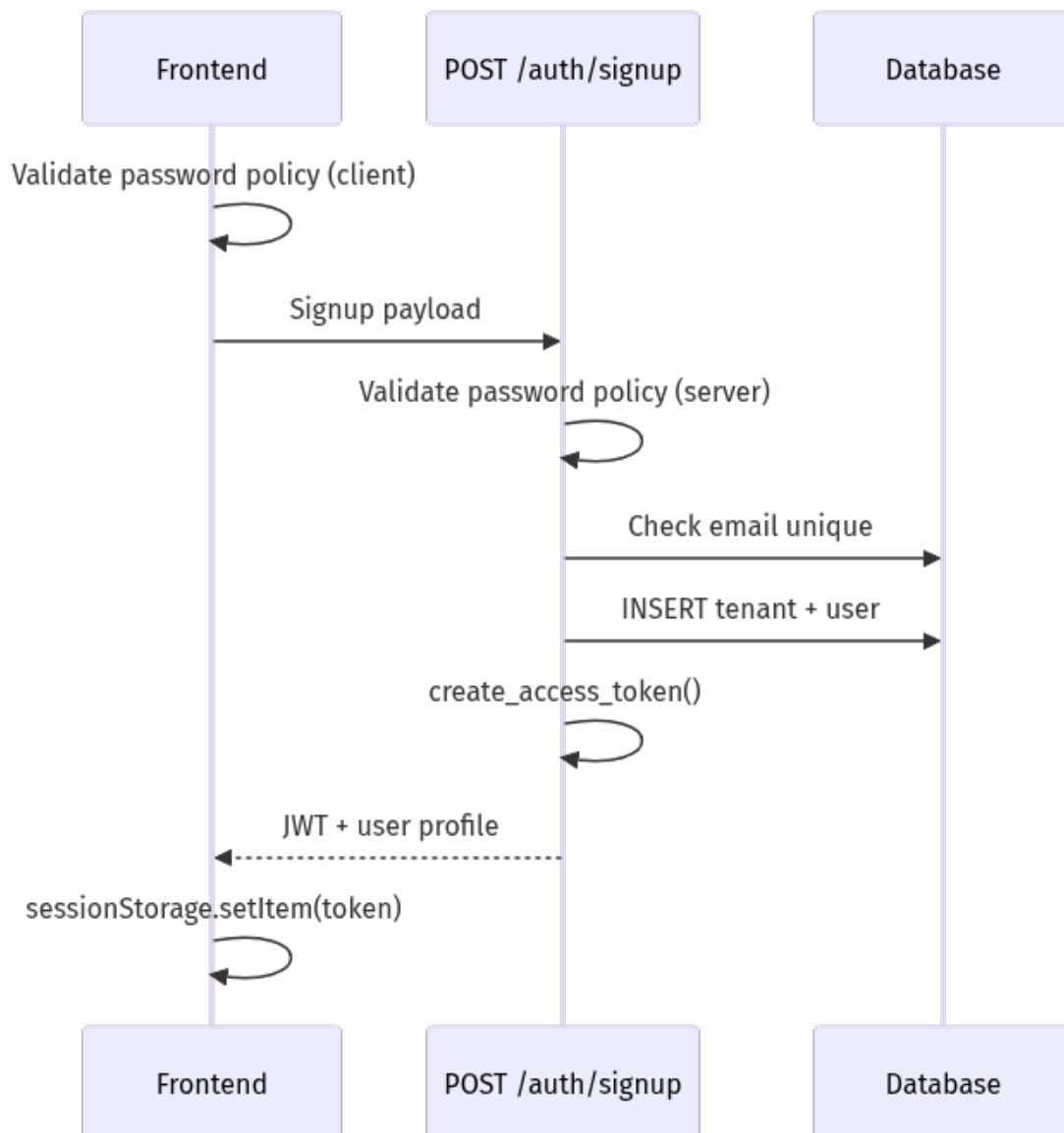
Field	Meaning
<code>total_items</code>	Checklist item count
<code>done_items</code>	Items with status <code>done</code>
<code>high_priority_items</code>	High-priority total
<code>high_priority_done_items</code>	High-priority completed
<code>medium_priority_items</code> , <code>low_priority_items</code>	Priority breakdown
<code>recommended_rule_count</code>	Auto-recommended rules

`get_latest_assessment` recomputes summary from live checklist JSON via

`build_assessment_summary()` to avoid stale counters.

12. Authentication and authorization

12.1 Signup flow



12.2 JWT payload

Claim	Content
<code>sub</code>	User ID
<code>tenant_id</code>	Organization ID

Claim	Content
<code>email</code>	User email
<code>exp</code>	Expiry (default 480 minutes / 8 hours)

Algorithm: `HS256`

Secret: `REGCHECK_JWT_SECRET_KEY` (must change in production)

12.3 Authorization model

Resource	Scope
GDPR assessments	<code>created_by_user_id</code> must match current user
Tenant	Implicit via user record; no cross-tenant access
Catalog (rules, items)	Shared read-only seed data

Historical note: Before 16-05-2026, assessments were tenant-scoped only; now creator-scoped so multiple users per tenant would not see each other's audits (multi-user per tenant not yet a product feature).

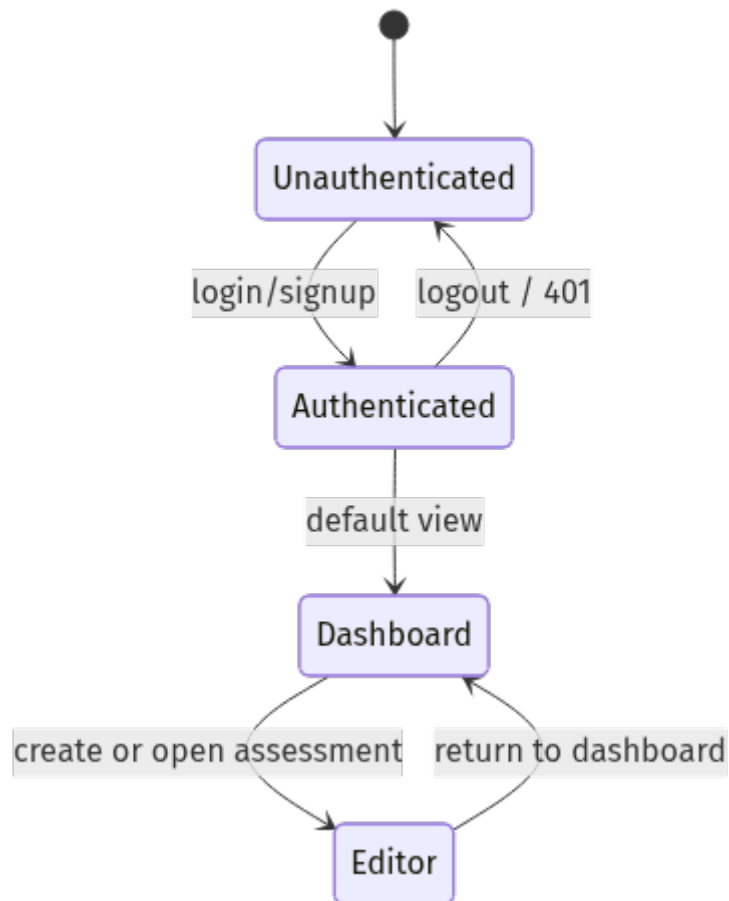
12.4 Password policy (aligned frontend + backend)

Rule	Value
Minimum length	12
Maximum length	128
Character classes	Upper, lower, digit, symbol
Spaces	Not allowed

Files: `backend/app/auth/password_policy.py` , `frontend/src/auth/password-policy.ts`

13. Frontend application

13.1 Application states



13.2 Component catalog

Component / module	Responsibility
<code>App.tsx</code>	Auth gate; routes dashboard vs editor; sidebar state
<code>auth-page.tsx</code>	Login/signup forms, policy hints
<code>auth-provider.tsx</code>	Token restore, <code>getCurrentUser</code> , unauthorized handler
<code>home-content.tsx</code>	Hero, <code>AssessmentDashboard</code> , metrics, loading states
<code>assessment-card.tsx</code>	History list cards with progress
<code>gdpr-playground.tsx</code>	Rule selection, live assessment, save, export

Component / module	Responsibility
<code>live-backend-input-sidebar.tsx</code>	Company profile and operational toggles
<code>checklist-item.tsx</code>	Status buttons, evidence draft/save UI
<code>saved-evidence-url-row.tsx</code>	Render persisted evidence links
<code>app-shell.tsx</code>	Navbar, footer, page layout
<code>navbar-dropdown.tsx</code>	Language and theme compact menus
<code>use-home-page-data.ts</code>	Bootstrap rule selector on load
<code>regcheck-api.ts</code>	Typed fetch wrapper, token management, <code>updateAssessment</code> , <code>exportAssessment</code>
<code>api-base-url.ts</code>	Resolves dev direct URL vs same-origin <code>/api</code> proxy
<code>report-export.ts</code>	Client-side HTML report download
<code>assessment-metrics.ts</code>	<code>done / total</code> formatting helpers
<code>evidence-url.ts</code>	Normalize URLs (<code>https://</code> prefix)

13.3 Live dashboard pattern

From `[[06-live-dashboard-and-report-export]]`:

1. User changes checkboxes / profile in sidebar
2. Frontend builds `liveAssessment` locally (mirrors backend checklist shape)
3. Metrics panel updates immediately
4. **Save** triggers `POST /gdpr/assessments` for new snapshots or `PATCH /gdpr/assessments/{id}` when editing an existing assessment; per-item status/evidence uses `PATCH .../checklist-items/{id}`
5. HTML export uses live state; CSV/Markdown require a saved assessment (`GET .../export`)

13.4 Theming

- `theme-provider.tsx` + `theme-toggle.tsx` — light/dark via CSS variables
- Tokyo Night aesthetic referenced in Obsidian todo notes
- Full viewport width layout (no max-width constraint on main content since 14-05-2026)

14. User journeys

14.1 First-time user

1. Open `http://localhost:3001`
2. Sign up with enterprise name and strong password
3. Land on **dashboard** (no assessments yet)
4. Click create assessment → **editor** opens
5. Fill company profile in sidebar; observe live checklist preview
6. Select/deselect rules; click generate/save
7. Mark items in progress; add evidence URL
8. Export HTML report
9. Return to dashboard — card shows progress `done / total`

14.2 Returning user with history

1. Log in (token restored from sessionStorage)
2. Dashboard lists assessments with dates and metrics
3. Click **View/Edit** on older assessment (not forced to latest)
4. Editor loads via `GET /gdpr/assessments/{id}`
5. PATCH updates sync dashboard on save

14.3 Consultant workflow

1. Create separate tenant per client (signup per enterprise name)
 2. Run intake per client; save assessment
 3. Export HTML per engagement
 4. Use evidence URLs as lightweight audit trail
-

15. Internationalization

Implemented 14-05-2026; Spanish added subsequently.

15.1 Stack

- `i18next` + `react-i18next`
- `i18next-browser-languagedetector` — `localStorage` key `regcheck-locale`
- `LocaleSync` — sets `document.documentElement.lang`

15.2 Namespaces

Namespace	Contents
<code>common</code>	Brand, nav, footer, theme, shared actions
<code>auth</code>	Login/signup, password policy strings
<code>home</code>	Hero, dashboard, metrics, loading
<code>playground</code>	Sidebar, checklist, assessment output
<code>errors</code>	Bootstrap failures, toasts

15.3 Locales

Code	Path	Status
<code>en</code>	<code>frontend/src/i18n/locales/en/</code>	Complete
<code>es</code>	<code>frontend/src/i18n/locales/es/</code>	Complete for shipped namespaces

15.4 What is not translated

GDPR rule labels, checklist titles, and descriptions come from the **API/seed in English**. Translating catalog content would require backend i18n or duplicate seed rows per locale (future work).

15.5 Usage pattern

```
const { t } = useAppTranslation("home");
return <h1>{t("hero.title")}</h1>;
```

Date/number formatting: `formatDateTime`, `formatNumber` in `i18n/format.ts` (`en` → `en-GB` Intl locale).

16. Report export

16.1 Mechanisms

Format	Where	Requirement
HTML	<code>frontend/src/lib/report-export.ts</code> — browser <code>Blob</code> download	Works on live/draft state
CSV / Markdown	<code>GET /api/v1/gdpr/assessments/{id}/export?format=</code>	Saved assessment only; frontend <code>exportAssessment()</code>

- HTML filename: `regcheck-report-<assessment_id>.html` — user prints to PDF via browser
- Server export filenames: `regcheck-assessment-<id>.csv` / `.md`

16.2 Report sections (academic alignment)

Per `[[06-live-dashboard-and-report-export]]`, exported documents include:

1. Portada y contexto (cover and context)
2. Índice (table of contents)
3. Resumen ejecutivo (executive summary)
4. Catálogo y recomendación (catalog and recommendations)
5. Problema y justificación (problem and justification)
6. Arquitectura técnica (technical architecture)
7. Proceso con evidencias (process with evidence)
8. Guía de despliegue (deployment guide)

- 9. Manual de uso (user manual)
- 10. Checklist generado (generated checklist)
- 11. Historial y roadmap (history and roadmap)

16.3 Data sources for export

Section	Source
Checklist + status	<code>liveAssessment</code> frontend state
Evidence	Per-item <code>evidence_entries</code> in snapshot
History	Local assessment history list
Metrics	<code>AssessmentSummary</code>

17. Configuration and environment

17.1 Frontend (`regcheck/.env.example`)

Variable	Default	Used by
<code>FRONTEND_PORT</code>	3001	Docker port mapping
<code>BACKEND_PORT</code>	8000	Documented; backend runs in other repo
<code>VITE_API_BASE_URL</code>	<code>http://localhost:8000</code>	Vite dev / preview — browser calls backend directly
<code>REGCHECK_API_BASE_URL</code>	<code>http://localhost:8000</code>	nginx <code>/api</code> upstream in Docker (<code>make up</code>)

On **Railway**, set `REGCHECK_API_BASE_URL` to the backend service URL (e.g. `https://regcheck-backend-production.up.railway.app`). The SPA uses same-origin `/api` ; do not set the browser-facing build to the backend URL.

17.2 Backend (`regcheck-backend/.env.example`)

Variable	Default	Used by
<code>BACKEND_PORT</code>	8000	Uvicorn / Docker
<code>POSTGRES_*</code>	regcheck	Docker Compose Postgres
<code>REGCHECK_DATABASE_URL</code>	PostgreSQL DSN in Docker	SQLAlchemy
<code>DEV_BACKEND_DATABASE_URL</code>	<code>sqlite:///./regcheck.db</code>	<code>make dev</code> local SQLite
<code>REGCHECK_FRONTEND_ORIGIN</code>	<code>http://localhost:3001</code>	CORS primary origin
<code>REGCHECK_CORS_ORIGINS</code>	localhost:3001, 127.0.0.1:3001	CORS CSV list
<code>REGCHECK_JWT_SECRET_KEY</code>	<code>change-me-in-production</code>	JWT signing

Railway injects `DATABASE_URL` / `DATABASE_PRIVATE_URL` ; the app normalizes `postgres://` → `postgresql+psycpg://` when `REGCHECK_DATABASE_URL` is unset.

17.3 Backend settings (`REGCHECK_` prefix)

Setting	Default	Description
<code>REGCHECK_DEBUG</code>	false	Debug mode
<code>REGCHECK_DATABASE_URL</code>	<code>sqlite:///./regcheck.db</code>	SQLAlchemy URL (overridden by Railway <code>DATABASE_URL</code>)
<code>REGCHECK_DB_ECHO</code>	false	SQL echo
<code>REGCHECK_FRONTEND_ORIGIN</code>	<code>http://localhost:3001</code>	CORS
<code>REGCHECK_CORS_ORIGINS</code>	localhost origins	Extra CORS origins (CSV)
<code>REGCHECK_CORS_ORIGIN_REGEX</code>	<code>*.up.railway.app</code>	Optional regex CORS
<code>REGCHECK_JWT_SECRET_KEY</code>	change-me...	JWT secret
<code>REGCHECK_JWT_ALGORITHM</code>	HS256	JWT algorithm
<code>REGCHECK_JWT_EXPIRE_MINUTES</code>	480	Token TTL

`extra="ignore"` on settings — non-prefixed keys in `.env` do not break startup.

18. Development guide

18.1 Prerequisites

- **Frontend:** Node.js 22+, pnpm 11+
- **Backend:** Python 3.12+
- Docker & Docker Compose (optional, per repo)

18.2 First-time setup (two repos)

Backend (`regcheck-backend`):

```
cd regcheck-backend
cp .env.example .env
make install
make ci           # compileall + pylint
make dev          # http://localhost:8000 - SQLite regcheck.db
```

Frontend (`regcheck`):

```
cd regcheck
cp .env.example .env
make install
make ci           # typecheck + lint + build
make dev          # http://localhost:3001 - requires backend on :8000
```

18.3 Daily development loop (recommended)

Run both services in separate terminals:

```
# Terminal 1
cd regcheck-backend && make dev

# Terminal 2
cd regcheck && make dev
```

Swagger: `http://localhost:8000/api/docs` . Frontend: `http://localhost:3001` .

18.4 Make targets — `regcheck` (frontend)

Target	Description
<code>install</code>	<code>pnpm install</code> in <code>frontend/</code>
<code>dev</code>	Vite on <code>:3001</code>
<code>preview</code>	Production build + Vite preview
<code>dev-docker</code>	Hot-reload frontend container
<code>up</code>	Production-like nginx container (backend on host <code>:8000</code>)
<code>stop</code> / <code>down</code>	Stop / remove frontend containers
<code>build</code>	Vite production build
<code>build-docker</code>	Build frontend image
<code>typecheck</code> / <code>lint</code>	TypeScript quality
<code>ci</code>	typecheck + lint + build
<code>logs</code> / <code>clean</code> / <code>ensure-env</code>	Ops helpers

18.5 Make targets — `regcheck-backend`

Target	Description
<code>install</code>	Python venv + pip requirements
<code>dev</code>	Uvicorn <code>--reload</code> , SQLite
<code>dev-docker</code> / <code>up</code>	Postgres + backend containers
<code>stop</code> / <code>down</code>	Stop / remove backend stack
<code>build</code>	<code>compileall</code> backend
<code>pylint</code>	Lint all Python modules
<code>ci</code>	build + pylint
<code>logs</code> / <code>clean</code> / <code>ensure-env</code>	Ops helpers

18.6 Git workflow scripts

In **regcheck** only, `vendor/scripts/git/` (submodule): `push_to_origin.sh`, `merge_to_dev.sh`, `merge_dev_to_main.sh`, etc.

19. Docker and deployment

19.1 Railway (current production)

Service	Repo	URL	Notes
Frontend	<code>regcheck</code>	<code>https://regcheck-production.up.railway.app/</code>	<code>REGCHECK_API_BASE_URL</code> → backend; nginx proxies <code>/api</code>
Backend	<code>regcheck-backend</code>	<code>https://regcheck-backend-production.up.railway.app/</code>	Managed Postgres via <code>DATABASE_URL</code> ; Swagger at <code>/api/docs</code>

Each repo includes `railway.json` pointing at its `Dockerfile`.

19.2 Docker Compose — `regcheck-backend`

Service	Image / build	Port	Depends on
postgres	postgres:16-alpine	internal	—
backend	<code>backend/Dockerfile</code>	8000	postgres healthy

Health: `pg_isready`; backend GET `/api/v1/health`. Dev overlay: bind-mount `backend/`, `uvicorn --reload`.

19.3 Docker Compose — `regcheck` (frontend only)

Service	Build	Port	Notes
frontend	<code>frontend/Dockerfile</code>	3001	Proxies <code>/api/</code> to <code>REGCHECK_API_BASE_URL</code> (rewrites <code>localhost</code> → <code>host.docker.internal</code> in container)

Health: wget homepage. Dev overlay: Vite dev server in container.

19.4 Production frontend image

1. `pnpm install --frozen-lockfile`
2. `pnpm run build` (`VITE_API_BASE_URL` empty in production — same-origin `/api`)
3. `docker-entrypoint.sh` renders `nginx.conf.template` and sets `runtime-config.js`
4. nginx serves `dist/` and proxies `/api/` to the backend upstream

19.5 Hetzner deployment (academic target)

Railway satisfies interim hosting. For the subject's Hetzner requirement:

1. Provision Ubuntu VPS on Hetzner Cloud
2. Install Docker + Compose
3. Clone **both** repositories; configure `.env` with strong secrets
4. `make up` in `regcheck-backend`, then `make up` in `regcheck` (or a single VPS compose overlay)
5. Reverse proxy (Caddy/nginx) with TLS in front of :3001
6. Point DNS to VPS; verify health and signup on the public URL

20. Quality assurance and CI

20.1 Local gates

Repo	Command	Validates
<code>regcheck</code>	<code>make typecheck</code>	TypeScript types
<code>regcheck</code>	<code>make lint</code>	ESLint, zero warnings
<code>regcheck</code>	<code>make build</code>	Vite production build
<code>regcheck</code>	<code>make ci</code>	typecheck + lint + build
<code>regcheck-backend</code>	<code>make build</code>	Python <code>compileall</code>

Repo	Command	Validates
<code>regcheck-backend</code>	<code>make pylint</code>	Python style/errors
<code>regcheck-backend</code>	<code>make ci</code>	build + pylint

20.2 GitHub Actions (per repository)

`regcheck/.github/workflows/ci.yml` : checkout → pnpm 11 → Node 22 → `make install` → typecheck → lint → build

`regcheck-backend/.github/workflows/ci.yml` : checkout → Python 3.12 → `make install` → build → pylint

Triggers (both): push to `main` , `develop` , `fix/**` , `feat/**` ; PRs to `main` / `develop` . Concurrency cancels in-progress runs on the same ref.

20.3 Testing gap

Neither repository ships automated unit or integration tests. Regression relies on CI static analysis, manual smoke tests (`make dev` in both repos), and daily report verification notes.

20.4 Verification practices (from daily reports)

Typical verification before merging:

- Frontend: `make typecheck` , `make lint` , `make build`
- Backend: `make pylint` on touched modules, `make build`
- Manual smoke: backend startup + frontend login → save assessment → export HTML/CSV

21. Documentation map

Document	Description
<code>[[00-index]]</code>	Hub: MVP state, Submit 1 inputs/outputs
<code>[[01-sqlite-persistence-goal]]</code>	Why SQLite first

Document	Description
[[02-sqlite-schema-and-seeding]]	Tables and idempotent seed
[[03-fastapi-db-integration]]	Request flow, session dependency
[[04-validation-and-operations]]	Runbook
[[05-frontend-backend-integration]]	API client, CORS, tester flow
[[06-live-dashboard-and-report-export]]	Live preview + HTML export
[[Subject_pr1.EN]] / [[Subject_pr1.ES]]	Academic brief (EN/ES)
<code>docs/obsidian/reports/DD-MM-YYYY.md</code>	Daily engineering logs
<code>README.md</code>	Quick start, MVP definition
PROJECT-REPORT.md	This extended report

22. Development chronology

Detailed logs in `docs/obsidian/reports/` . Summary by date:

11-05-2026 — Checklist lifecycle and evidence MVP

- `PATCH` endpoint for checklist items
- Evidence metadata model (`label` , `reference_url` , `notes` , `created_at`)
- Dynamic follow-up questions from operational context
- Status updates in UI; export includes evidence
- Unsaved evidence draft indicator

12-05-2026 — Sidebar UX

- Fixed collapsed sidebar icon alignment (centered rail)

13-05-2026 — Authentication and tenancy

- JWT signup/login/logout/me

- Tenant created per signup; bcrypt password hashing
- All GDPR routes require auth; assessments have `tenant_id`
- Frontend auth gate; navbar user context

14-05-2026 — Internationalization and layout

- i18next namespaces (en); later es
- Full viewport width; sidebar-aware page body offset
- `NavbarDropdown` for language/theme
- `useAppTranslation` , `LocaleSync` , `formatDateTime`

15-05-2026 — Progress metrics

- `done / total` on dashboard and playground
- `done_items` , `high_priority_done_items` in summary
- Recompute summary from checklist on latest fetch

16-05-2026 — Password policy and ownership

- Strong password rules (12+ chars, complexity)
- Client + server policy enforcement
- `created_by_user_id` on assessments; creator-scoped access
- SQLite migration guard for existing DBs

18-05-2026 — Dashboard and evidence URLs

- Assessment dashboard after login; open any saved assessment by ID
- `GET /gdpr/assessments/{id}`
- Evidence URL normalization (`https://`); absolute links
- Local backend SQLite default; settings ignore stray `.env` keys
- Full-width dashboard when editor closed

19-05-2026 — Assessment update and server export

- `PATCH /gdpr/assessments/{id}` — regenerate snapshot while preserving status/evidence for unchanged items
- `GET /gdpr/assessments/{id}/export?format=csv|markdown`
- Frontend save uses PATCH for persisted assessments; sidebar CSV/Markdown export actions

Post-19-05-2026 — Split repositories and Railway

- **Frontend** and **backend** moved to separate Git repos (`regcheck` , `regcheck-backend`)
- Frontend nginx `/api` reverse proxy and `api-base-url.ts` for dev vs production routing
- Railway deployment for both services; CORS origin regex for `*.up.railway.app`
- CI split: frontend-only workflow in `regcheck` , backend-only in `regcheck-backend`

23. Academic compliance checklist

Use this when preparing the PDF submission:

#	Report section (Subject §5)	RegCheck artifact
1	Cover page	Export section 1 + this report header
2	Table of contents	This document TOC
3	Executive summary	§1
4	Problem and justification	§3
5	Technical architecture	§6, §9 diagrams
6	Development process with evidence	§22 + <code>reports/*.md</code> + screenshots
7	Deployment guide	§19
8	User manual	§14 user journeys
9	Conclusions	§27
10	Improvement roadmap	§24

Live demo checklist:

- [x] Signup/login works on Railway URLs (verify before evaluation)
- [] Create assessment with cloud + dev toggles
- [] Show live metric update without save
- [] Save, refresh, reopen from dashboard
- [] Add evidence URL, show external link behavior
- [] Export HTML and print to PDF

24. Roadmap — Practice 2 and beyond

24.1 Planned functionalities (minimum 5)

#	Feature	Description	Est. effort
1	AI compliance assistant	LLM explains checklist items and suggests evidence	3–4 weeks
2	DPIA wizard	Guided Data Protection Impact Assessment flow	2–3 weeks
3	Native PDF export	Server-side or client PDF without print dialog	1 week
4	File evidence upload	S3-compatible storage with virus scan hook	2 weeks
5	Multi-user tenant roles	Admin vs auditor vs contributor	2 weeks
6	NIS2 / ENS modules	Additional domain modes in catalog	4+ weeks
7	Regulatory change alerts	RSS/BOE monitor with AI summaries	3 weeks

24.2 Performance and scalability

- Move all environments to PostgreSQL with Alembic migrations
- Add Redis caching for rule selector payload
- Paginate assessment history beyond 50 items
- CDN for frontend static assets

24.3 Security improvements for the tool

- httpOnly secure cookies option vs Bearer tokens
- Refresh tokens and rotation
- Rate limiting on auth endpoints
- Email verification and MFA
- Content Security Policy headers on frontend
- Secret management (Vault / Hetzner secrets) for production

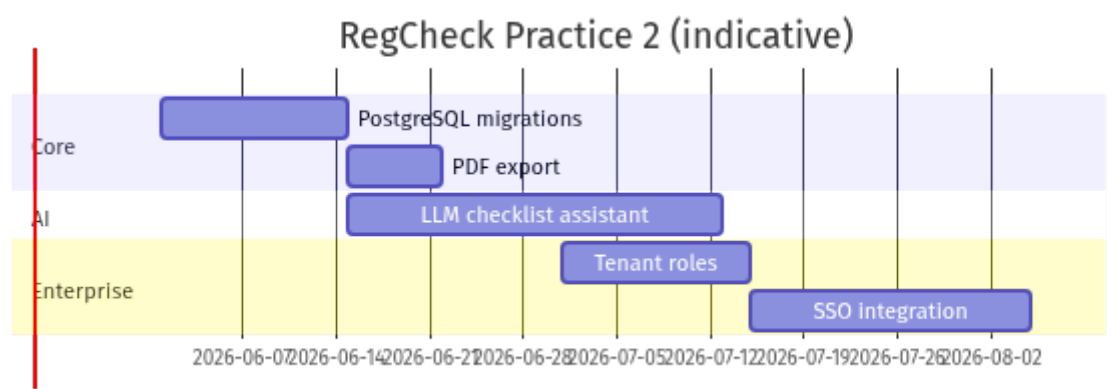
24.4 Integrations

- Google Drive / SharePoint evidence linking
- Jira/Linear ticket creation from checklist items
- SIEM webhook for SOC-related controls
- SSO (OIDC) for enterprise tenants

24.5 UX and product debt (near term)

Item	Source
Navbar section tracking	Review notes
Footer content improvement	Review notes
Color palette refresh	Review notes
Rename “Live Backend Input” section	Review notes
Hide ISO 27001 until supported	README / review
GitHub branch protection + required CI	Review notes
Formal compliance scoring model	Week 3 plan

24.6 Roadmap diagram



25. Security, risks, and limitations

25.1 Security controls implemented

- bcrypt password hashing (cost factor via gensalt)
- JWT expiration (8h default)
- Per-user assessment isolation
- Non-root container user for backend
- CORS origin allowlist
- SQL injection mitigation via SQLAlchemy/ORM parameterization

25.2 Risks

Risk	Impact	Mitigation path
XSS stealing JWT from sessionStorage	Account compromise	CSP, sanitize rendered URLs, shorten token TTL
Default JWT secret in dev	Token forgery if deployed carelessly	Enforce secret in production checklist
No rate limiting on login	Brute force	Add slowapi or reverse-proxy limits

Risk	Impact	Mitigation path
Metadata-only evidence	Weak audit proof	File upload + integrity hashes in P2
English-only catalog	Spanish users confused	Backend i18n or translated seed
SQLite local vs Postgres Docker	Environment drift	Single DB target + migration tests

25.3 Legal disclaimer

RegCheck output is **informational**. Organizations should validate controls with qualified legal and DPO review before claiming regulatory compliance.

26. Glossary

Term	Definition
GDPR	EU General Data Protection Regulation
RoPA	Record of Processing Activities
DSR	Data Subject Request
DPIA	Data Protection Impact Assessment
DPA	Data Processing Agreement
DevSecOps	Development + security + operations practices in SDLC
SOC	Security Operations Center
MVP	Minimum Viable Product
Assessment	Persisted snapshot of a checklist run
Catalog	Seeded rules and template checklist items
Tenant	Organization boundary in multi-tenant model
JWT	JSON Web Token used for API authentication
ENS	Esquema Nacional de Seguridad (Spanish security framework)

Term	Definition
NIS2	EU Network and Information Security Directive

27. Conclusion

RegCheck is a **functional GDPR compliance checklist MVP** with a clear split-repo architecture, documented development history, and production hosting on Railway while Hetzner remains the academic VPS target. The system successfully bridges the gap between regulatory abstraction and operational tasks through:

- A **deterministic, explainable rule engine** (not a black-box AI in v1)
- **Persisted assessments** with per-user history
- **Evidence metadata** and **exportable reports**
- A **modern developer experience** (split repos, Docker per service, CI, i18n, Railway deploy)

The codebase is structured for incremental enhancement: catalog expansion, AI assistance, stronger auth, automated tests, and Hetzner migration can proceed without rewriting the core assessment snapshot model.

Report version: 22-05-2026. Maintained in `regcheck/docs/obsidian/` ; covers both `regcheck` and `regcheck-backend` . Update when features, APIs, or deployment targets change.